

Growth Rates, Asymptotic Notation, and Case Analysis

Growth Rates, Asymptotic Notation, and Case Analysis

Algorithms — Student Handout

TOPIC 1 — Growth rates

1.1 A motivating question

Algorithm **A** takes $100n$ steps. Algorithm **B** takes n^2 steps. **Which one is faster?**

The answer is “it depends on n ”.

n	A: $100n$	B: n^2	Faster
10	1,000	100	B
50	5,000	2,500	B
100	10,000	10,000	tie
200	20,000	40,000	A
1,000	100,000	1,000,000	A
10,000	1,000,000	100,000,000	A

The crossover is exactly at $n = 100$, obtained by solving $100n = n^2$.

Which side of the crossover matters? The large- n side — because small inputs are fast on *any* algorithm. Nobody complains that a program is too slow on 10 items. This single observation justifies everything that follows: we study **behaviour as $n \rightarrow \infty$** , and we ignore constant factors.

1.2 Definition

Growth rate (order of growth) of an algorithm is the way its running time increases as a function of the input size n , considering only the dominant term and disregarding constant factors.

Three things we deliberately discard, and why:

We ignore	Why
Constant factors (the 100 in $100n$)	Machine-, compiler-, and language-dependent. A faster CPU changes the constant, never the shape.
Lower-order terms (the $5n$ in $3n^2 + 5n$)	Their contribution vanishes as n grows — see 1.3.
Small inputs	They finish fast regardless.

1.3 Why lower-order terms can be dropped

Take $T(n) = 3n^2 + 5n + 2$. The share of the total contributed by the $3n^2$ term:

n	$3n^2$	$5n$	2	Total $T(n)$	$3n^2$ as % of total
1	3	5	2	10	30.0%
10	300	50	2	352	85.2%
100	30,000	500	2	30,502	98.4%
1,000	3,000,000	5,000	2	3,005,002	99.83%
10,000	300,000,000	50,000	2	300,050,002	99.98%

The lower-order terms do not become *zero* — they become *irrelevant*. By $n = 1000$ the $5n + 2$ part accounts for less than 0.2% of the work. So we write $T(n) = \theta(n^2)$ and lose nothing that matters.

1.4 The standard hierarchy

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$

Function	Name	Typical example
1	constant	array index access, push/pop on a stack
$\log n$	logarithmic	binary search, balanced BST lookup
n	linear	linear search, finding the maximum
$n \log n$	linearithmic	merge sort, heapsort, quicksort (average)
n^2	quadratic	insertion / selection / bubble sort
n^3	cubic	naive matrix multiplication, Floyd-Warshall
2^n	exponential	subset enumeration, naive recursive Fibonacci
$n!$	factorial	brute-force travelling salesman, all permutations

1.5 The doubling test

If the input size doubles, what happens to the running time?

Growth	Double $n \rightarrow$ running time	Reasoning
$O(1)$	unchanged	independent of n
$O(\log n)$	plus one step	$\log(2n) = \log n + 1$
$O(n)$	doubles ($\times 2$)	linear
$O(n \log n)$	slightly more than doubles	$2n \cdot \log(2n) = 2n \log n + 2n$
$O(n^2)$	$\times 4$	$(2n)^2 = 4n^2$
$O(n^3)$	$\times 8$	$(2n)^3 = 8n^3$
$O(2^n)$	gets squared	$2^{2n} = (2^n)^2$

The last row is qualitatively different from the others. Doubling the input does not double an exponential algorithm's runtime — it *squares* it.

1.6 Concrete magnitudes

Assume a machine performing **10^9 operations per second**.

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3 ns	10 ns	33 ns	100 ns	1 μ s	1 μ s	3.6 ms
20	4 ns	20 ns	86 ns	400 ns	8 μ s	1 ms	77 years
50	6 ns	50 ns	282 ns	2.5 μ s	125 μ s	13 days	$\gg$ age of universe
100	7 ns	100 ns	664 ns	10 μ s	1 ms	4×10^{13} yr	$\gg \gg$
1,000	10 ns	1 μ s	10 μ s	1 ms	1 s	—	—
10^6	20 ns	1 ms	20 ms	17 min	31,700 yr	—	—

Three observations:

- At $n = 50$, an exponential algorithm needs 13 days; a quadratic one needs 2.5 microseconds — a factor of roughly 4×10^{11} .
- At $n = 10^6$, $n \log n$ takes 20 ms while n^2 takes 17 minutes. This is why merge sort matters.

3. **Hardware cannot rescue a bad algorithm.** A computer $1,000\times$ faster turns 13 days into 19 minutes — but raising n from 50 to 60 puts you back past 13 days. The only real fix is a better algorithm.

1.7 Worked example

An $O(n^2)$ program processes 1,000 records in 4 seconds. Approximately how long will 5,000 records take?

The input scales by a factor of 5, so the time scales by $5^2 = 25 \rightarrow$ **100 seconds**.

If the same program were $O(n \log n)$, the factor would be $5 \times \log(5000)/\log(1000) \approx 5 \times 1.23 \approx 6.2 \rightarrow$ approximately **25 seconds**.

TOPIC 2 — Asymptotic functions and notation

2.1 What asymptotic notation is for

Asymptotic notation is the mathematical machinery for saying “**eventually, and ignoring constant factors.**”

Every symbol in the definitions below encodes one of those two phrases.

2.2 Big-O — upper bound

Definition

$O(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0 \}$

Read as: **f grows no faster than g .**

Symbol	Its job
c	absorbs constant factors — “ignoring constants”
n_0	ignores small inputs — “eventually”

In the figure below, $f(n)$ and $c \cdot g(n)$ may cross to the left of the dashed line at n_0 . To the right of n_0 , $f(n)$ never rises above $c \cdot g(n)$.

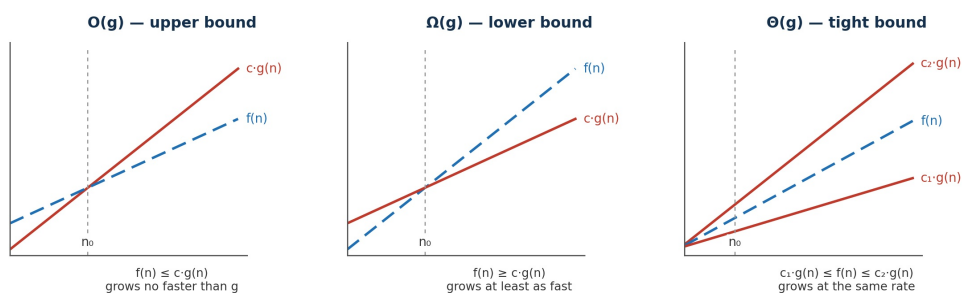


Figure 1 — the three asymptotic bounds. Curves may cross before n_0 ; the relationship is only required to hold for every $n \geq n_0$.

Worked proof 1 — a loose bound

Claim: $3n^2 + 5n + 2 = O(n^2)$

For all $n \geq 1$ we have $n \leq n^2$ and $1 \leq n^2$. Therefore:

$$3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$$

The definition is satisfied with $c = 10$, $n_0 = 1$. ■

Numerical check:

n	$f(n) = 3n^2 + 5n + 2$	$10n^2$	$f(n) \leq 10n^2?$
1	10	10	yes (equal)
2	24	40	yes
5	102	250	yes
10	352	1,000	yes
100	30,502	100,000	yes

Worked proof 2 — a tighter c

Claim: the same function is $O(n^2)$ with $c = 4$.

$$\begin{aligned}
 3n^2 + 5n + 2 &\leq 4n^2 \\
 \Leftrightarrow 5n + 2 &\leq n^2 \\
 \Leftrightarrow n^2 - 5n - 2 &\geq 0
 \end{aligned}$$

Roots: $n = (5 \pm \sqrt{(25 + 8)}) / 2 = (5 \pm 5.745) / 2 \rightarrow$ positive root ≈ 5.37 . The inequality therefore holds for all $n \geq 6$, giving $c = 4$, $n_0 = 6$.

Boundary check:

n	f(n)	$4n^2$	Holds?
5	102	100	NO
6	140	144	yes
7	184	196	yes

Note. It is the *pair* (c, n_0) that matters, not either constant alone. Shrinking c forces n_0 further out. Both proofs above are valid proofs of the same statement — the definition only requires that *some* such pair exists.

Worked disproof

Claim: $n^2 \neq O(n)$

Suppose it were. Then there exist $c, n_0 > 0$ with $n^2 \leq c \cdot n$ for all $n \geq n_0$. Dividing by $n > 0$ gives $n \leq c$ for all $n \geq n_0$. Now choose $n = \max(n_0, c) + 1$. This n satisfies $n \geq n_0$, yet $n > c$ — a contradiction. ■

2.3 Big-Omega — lower bound

Definition

$$\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0 \}$$

Read as: **f grows at least as fast as g.**

Worked example

Claim: $3n^2 + 5n + 2 = \Omega(n^2)$

Since $5n \geq 0$ and $2 > 0$ for all $n \geq 1$:

$$3n^2 + 5n + 2 \geq 3n^2$$

So $c = 3$, $n_0 = 1$. ■

Also true, but weak: $3n^2 + 5n + 2 = \Omega(n)$ and $= \Omega(1)$. These are correct statements that carry almost no information — which illustrates the difference between a bound that is merely **valid** and one that is **tight**.

2.4 Big-Theta — tight bound

Definition

$\theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0 \}$

Read as: **f grows at exactly the same rate as g.**

Theorem. $f(n) = \theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Worked example 1

Combining 2.2 and 2.3: $3n^2 + 5n + 2 = \theta(n^2)$ with $c_1 = 3$, $c_2 = 10$, $n_0 = 1$.

n	$3n^2$ (lower)	f(n)	$10n^2$ (upper)	Sandwiched?
1	3	10	10	yes
5	75	102	250	yes
10	300	352	1,000	yes
100	30,000	30,502	100,000	yes

Worked example 2 — a function with a subtraction

Claim: $\frac{1}{2}n^2 - 3n = \theta(n^2)$

Upper bound. $\frac{1}{2}n^2 - 3n \leq \frac{1}{2}n^2$ holds for all $n \geq 0$, since a non-negative quantity is being subtracted. $\rightarrow c_2 = \frac{1}{2}$

Lower bound. We need $\frac{1}{2}n^2 - 3n \geq c_1 n^2$. Dividing through by n^2 :

$$\frac{1}{2} - 3/n \geq c_1$$

The left side increases with n. At $n = 7$: $\frac{1}{2} - 3/7 = 0.5 - 0.4286 = 0.0714 = 1/14$. $\rightarrow c_1 = 1/14$, $n_0 = 7$

Check:

n	$(1/14)n^2$	$\frac{1}{2}n^2 - 3n$	$\frac{1}{2}n^2$	OK?
6	2.57	0	18	lower bound fails ($0 < 2.57$)
7	3.5	3.5	24.5	yes (equality at the boundary)
10	7.14	20	50	yes
20	28.6	140	200	yes

■ with $c_1 = 1/14$, $c_2 = \frac{1}{2}$, $n_0 = 7$.

2.5 Little-o and little-omega

Notation	Definition	Meaning	Example
$o(g)$	for every $c > 0$ there is an n_0 with $f(n) < c \cdot g(n)$	strictly slower	$2n = o(n^2)$ but $2n^2 \neq o(n^2)$
$\omega(g)$	for every $c > 0$ there is an n_0 with $f(n) > c \cdot g(n)$	strictly faster	$n^2/2 = \omega(n)$

The distinction: in θ the bound must hold for **some** c; in o it must hold for **all** c. By analogy, θ behaves like $\leq$ and o behaves like $<$.

2.6 The limit method

Compute $L = \lim_{n \rightarrow \infty} f(n)/g(n)$:

L	Conclusion
θ	$f = O(g)$ and $f = o(g)$ — f is strictly smaller
finite, $\neq \theta$	$f = \theta(g)$
∞	$f = \Omega(g)$ and $f = \omega(g)$ — f is strictly larger

Example. $f = 3n^2 + 5n + 2$, $g = n^2$

$$\lim (3n^2 + 5n + 2)/n^2 = \lim (3 + 5/n + 2/n^2) = 3$$

Finite and non-zero $\rightarrow \theta(n^2)$, confirming section 2.4 in a single line.

Example. $f = n$, $g = n^2 \rightarrow \lim n/n^2 = \lim 1/n = 0 \rightarrow n = o(n^2)$.

2.7 Useful properties

Property	Statement
Transitivity	$f = O(g)$ and $g = O(h) \implies f = O(h)$ (holds for Θ , Ω , o , ω too)
Reflexivity	$f = O(f)$, $f = \Omega(f)$, $f = \Theta(f)$
Symmetry	$f = O(g) \iff g = \Theta(f)$
Transpose symmetry	$f = O(g) \iff g = \Omega(f)$
Sum rule	$O(f) + O(g) = O(\max(f, g))$ — applies to sequential code blocks
Product rule	$O(f) \times O(g) = O(f \times g)$ — applies to nested loops

Applied. A $\theta(n)$ loop followed by a $\theta(n^2)$ loop gives $\theta(n + n^2) = \theta(n^2)$ by the sum rule. A $\theta(n)$ loop *nested inside* a $\theta(n)$ loop gives $\theta(n \times n) = \theta(n^2)$ by the product rule.

2.8 Common misconceptions

1. **“O means worst case.”** False. O is a *bound*; worst case is a *case*. The two are independent — see Topic 3.
2. **“ $f(n) = O(g(n))$ is an equation.”** It is not. $O(g)$ is a *set*, and $=$ here means $\in$. That is why it never reverses: $3n = O(n^2)$ is fine, $O(n^2) = 3n$ is meaningless.
3. **“ $O(n^2)$ means the algorithm takes n^2 steps.”** No — it is a ceiling. $n = O(n^2)$ is a true statement, merely an uninformative one.
4. **“Big-O tells you which program is faster.”** It tells you which one *scales* better. For small n , the constant factors can easily decide the race the other way.

TOPIC 3 — Best-case and worst-case time complexity

3.1 Two independent axes

AXIS 1 — WHICH INPUT? (pick one input of size n)		AXIS 2 — WHICH BOUND? (describe that function)
<pre> +-----+ Best case Worst case Average case +-----+ </pre>	x	<pre> +-----+ O upper Omega lower Theta tight +-----+ </pre>

Choosing on Axis 1 gives you a **function of n** . Choosing on Axis 2 **describes** that function. The two choices are made independently, so all nine combinations are grammatical. “*The worst case is $\Omega(n \log n)$* ” and “*the best case is $O(n)$* ” are both perfectly legal statements.

This is precisely why “*Big-O means worst case*” is wrong.

Definitions

Case	Definition
Best case	The minimum running time over all inputs of size n — the most favourable input.
Worst case	The maximum running time over all inputs of size n — the least favourable input.
Average case	The expected running time over a stated probability distribution of inputs of size n .

The average case is meaningless without naming the distribution. “Average” normally means uniformly random input, and that assumption is often unrealistic in practice.

3.2 Worked example — linear search

```

LINEAR-SEARCH(A, n, x)
  for i = 1 to n
    if A[i] == x
      return i
  return NIL

```

For a concrete array of $n = 8$ elements, counting comparisons:

Situation	Comparisons	Complexity
x is at A[1]	1	$\theta(1)$ — best case
x is at A[4]	4	—
x is at A[8]	8	$\theta(n)$ — worst case (present)
x is absent	8	$\theta(n)$ — worst case

Average case, assuming x is present and equally likely at any of the 8 positions:

$$(1 + 2 + 3 + 4 + 5 + 6 + 7 + 8) / 8 = 36 / 8 = 4.5$$

In general $(1 + 2 + \dots + n)/n = (n+1)/2$, which is still $\theta(n)$.

The average case here is half the worst case — a constant factor of 2 — so asymptotically the two fall in the same class. Constants distinguish them; growth rate does not.

3.3 Worked example — insertion sort

```

INSERTION-SORT(A, n)
1  for j = 2 to n
2    key = A[j]
3    i = j - 1
4    while i > 0 and A[i] > key
5      A[i+1] = A[i]
6      i = i - 1
7    A[i+1] = key

```

Let t_j denote the number of times the while-loop **test** on line 4 executes for that value of j .

Line	Cost	Times executed
1	c_1	n
2	c_2	$n - 1$
3	c_3	$n - 1$
4	c_4	$\sum_{j=2}^n t_j$
5	c_5	$\sum_{j=2}^n (t_j - 1)$
6	c_6	$\sum_{j=2}^n (t_j - 1)$
7	c_7	$n - 1$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum t_j + c_5 \sum (t_j - 1) + c_6 \sum (t_j - 1) + c_7(n-1)$$

Best case — array already sorted

The test $A[i] > \text{key}$ fails immediately every time, so $t_j = 1$ for every j .

$$\sum_{j=2}^n t_j = n - 1 \quad \sum_{j=2}^n (t_j - 1) = 0$$

$$\begin{aligned} T(n) &= c_1 n + (c_2 + c_3 + c_4 + c_7)(n - 1) \\ &= an + b \quad (\text{linear in } n) \end{aligned}$$

→ $\theta(n)$

Worst case — array in reverse sorted order

Element $A[j]$ must travel past all $j - 1$ elements before it, so $t_j = j$.

$$\begin{aligned} \sum_{j=2}^n j &= n(n+1)/2 - 1 \\ \sum_{j=2}^n (j - 1) &= n(n-1)/2 \end{aligned}$$

$$T(n) = an^2 + bn + c \quad (\text{quadratic in } n)$$

→ $\theta(n^2)$

Numerical verification at $n = 8$

Input	Total while-tests $\sum t_j$	Total shifts $\sum (t_j - 1)$
Sorted [1..8]	7	0
Reversed [8..1]	$2+3+4+5+6+7+8 = 35$	$1+2+\dots+7 = 28$

Against the formulas: $n(n+1)/2 - 1 = 8 \cdot 9/2 - 1 = 35$ ✓ and $n(n-1)/2 = 8 \cdot 7/2 = 28$ ✓

Average case

Each element travels roughly halfway back, so $t_j \approx j/2$. Halving a quadratic leaves a quadratic → still $\theta(n^2)$.

3.4 Contrast — selection sort has no case distinction

```

SELECTION-SORT(A, n)
  for i = 1 to n - 1
    min = i
    for j = i + 1 to n
      if A[j] < A[min]
        min = j
    exchange A[i] with A[min]

```

Comparisons: $\sum_{i=1}^{n-1} (n - i) = (n-1) + (n-2) + \dots + 1 = n(n-1)/2$

At $n = 8$: $8 \times 7 / 2 = 28$ comparisons — **regardless of the input**, because no comparison is ever skipped on the basis of the data.

→ best = worst = average = $\theta(n^2)$

Compare this with insertion sort. Insertion sort spans $\theta(n)$ to $\theta(n^2)$; selection sort is fixed at $\theta(n^2)$. On nearly-sorted data insertion sort wins outright — a fact that is invisible unless the cases are analysed separately.

3.5 Why worst case is the default

Reason	Explanation
It is a guarantee	An upper bound that holds no matter what input arrives. The best case guarantees nothing.
Adversary-proof	In security-sensitive code an attacker will <i>deliberately</i> construct the worst input. Hash-collision denial-of-service attacks work exactly this way.
The worst case is often common	Searching for a record that is not in the database hits the worst case every single time.
The average case requires assumptions	A distribution must be stated, and real input is rarely uniformly random.

3.6 Summary

Algorithm	Best	Worst	Average	Cases differ?
Linear search	$\theta(1)$	$\theta(n)$	$\theta(n)$	yes
Binary search	$\theta(1)$	$\theta(\log n)$	$\theta(\log n)$	yes
Insertion sort	$\theta(n)$	$\theta(n^2)$	$\theta(n^2)$	yes
Selection sort	$\theta(n^2)$	$\theta(n^2)$	$\theta(n^2)$	no
Bubble sort (with early-exit flag)	$\theta(n)$	$\theta(n^2)$	$\theta(n^2)$	yes
Merge sort	$\theta(n \log n)$	$\theta(n \log n)$	$\theta(n \log n)$	no
Quicksort	$\theta(n \log n)$	$\theta(n^2)$	$\theta(n \log n)$	yes

Selection sort and merge sort show no case distinction because neither algorithm's control flow depends on the data: selection sort always performs every comparison, and merge sort always splits down the middle regardless of the contents.

Exercises

1. Prove $2n^2 + 3n + 1 = O(n^2)$.
2. Prove $n^2 + n \neq O(n)$.
3. Is $2^{n+1} = O(2^n)$?
4. Is $2^{2^n} = O(2^n)$?
5. Find c_1, c_2, n_0 showing $n^2/2 - 2n = \theta(n^2)$.
6. An $O(n \log n)$ algorithm sorts 10^6 items in 2 seconds. Estimate the time for 4×10^6 items.
7. State the best and worst case of bubble sort with an early-exit flag, and explain why adding the flag does not change the worst case.
8. True or false: "If $f(n) = O(n^2)$ then $f(n) = O(n^3)$."

Solutions

1. For $n \geq 1$: $3n \leq 3n^2$ and $1 \leq n^2$, so the expression is $\leq 6n^2$. Take **$c = 6$, $n_0 = 1$** .
2. Suppose $n^2 + n \leq cn$ for all $n \geq n_0$. Dividing by n gives $n + 1 \leq c$. Choosing $n = \max(n_0, c)$ yields a contradiction.

3. Yes. $2^{n+1} = 2 \cdot 2^n$, so $c = 2$, $n_0 = 1$ works.

4. No. $2^{2n} = (2^n)^2$, and the ratio $2^{2n}/2^n = 2^n \rightarrow \infty$. Contrast with Q3: coefficients can be absorbed by c , exponents cannot.

5. Upper: $\leq n^2/2$, so $c_2 = 1/2$. Lower: $\frac{1}{2} - 2/n \geq c_1$; at $n = 8$, $\frac{1}{2} - \frac{1}{4} = \frac{1}{4}$. So $c_1 = 1/4$, $n_0 = 8$.

6. Ratio = $4 \times \log(4 \times 10^6) / \log(10^6) = 4 \times 21.9 / 19.9 \approx 4.4 \rightarrow$ approximately **8.8 seconds**.

7. Best $\theta(n)$ — a single clean pass over already-sorted input. Worst $\theta(n^2)$. On reverse-sorted input every pass performs at least one swap, so the flag never triggers an early exit.

8. True. O is an upper bound, and any function bounded by cn^2 is also bounded by cn^3 for $n \geq 1$. The bound is valid but not tight.